

✓ Aula 4 - Análise Preditiva e Classificativa: Análise de Sentimentos

Contextualização

Na aula anterior, exploramos os fundamentos de **Mineração de Dados** e **Processamento de Linguagem Natural (NLP)**. Discutimos como extrair conhecimento a partir de dados textuais não estruturados e conhecemos algumas técnicas fundamentais de pré-processamento de texto, como:

- **Tokenização**
- **Remoção de Stopwords**
- **Lematização**
- **Vetorização com Bag-of-Words e TF-IDF**

Hoje, daremos continuidade a esse tema com uma aplicação prática e extremamente comum no mundo real: **Análise de Sentimentos**.

Objetivos da Aula

- Compreender o conceito de **análise de sentimentos**
- Trabalhar com dados reais de avaliações de filmes (IMDb Dataset)
- Aplicar técnicas de pré-processamento de texto
- Construir um modelo preditivo para **classificar sentimentos**
- Avaliar o desempenho do modelo

O que é Análise de Sentimentos?

A **análise de sentimentos** é uma tarefa de **classificação de texto** que busca identificar a **opinião ou emoção** expressa em um texto. Essa análise é amplamente utilizada em áreas como:

- Monitoramento de redes sociais
- Avaliação de satisfação de clientes
- Feedbacks de produtos
- Opiniões políticas
- Análises de mercado

Normalmente, os sentimentos são classificados em:

- **Positivo**
- **Negativo**
- (Opcionalmente) **Neutro**

Dataset Utilizado

Utilizaremos o **IMDb Dataset**, que contém **50.000 resenhas de filmes** em inglês, rotuladas com os sentimentos **positivos** ou **negativos**.

Colunas:

- `review`: texto da resenha
- `sentiment`: rótulo do sentimento associado à resenha (`positive` ou `negative`)

Vamos começar?

Nesta aula, você aprenderá a construir um pipeline completo de análise de sentimentos, utilizando Python e bibliotecas como:

- `pandas` e `matplotlib` (análise exploratória)
- `nltk` ou `scikit-learn` (pré-processamento e vetorização)
- `LogisticRegression`, `Naive Bayes` ou `SVM` (classificação)
- `metrics` (avaliação do modelo)

Leitura do Dataset e Análise Exploratória Inicial

Vamos começar lendo o dataset e entendendo sua estrutura básica.

Carregando o dataset

```
import pandas as pd
from google.colab import files
```

```
# Faz o upload manual do arquivo (você deve selecionar o arquivo TMDb Dataset.csv)
```

https://colab.research.google.com/drive/1XRYAJZdpd7IBdnR0ixYA_qbYj1QY1-bB#scrollTo=Fps8CtbBGzSD&printMode=true

```
# Faz upload manual do arquivo (pode ser automático o upload dos datasets),
uploaded = files.upload()

# Certifique-se de usar o nome exato do arquivo enviado (incluindo espaços)
df = pd.read_csv("IMDB Dataset.csv")

# Visualizar as primeiras 5 linhas
df.head()
```

 Escolher arquivos IMDB Dataset.csv

- **IMDB Dataset.csv**(text/csv) - 66212309 bytes, last modified: 19/10/2019 - 100% done
Saving IMDB Dataset.csv to IMDB Dataset.csv

	review	sentiment	
0	One of the other reviewers has mentioned that ...	positive	
1	A wonderful little production. The...	positive	
2	I thought this was a wonderful way to spend ti...	positive	
3	Basically there's a family where a little boy ...	negative	
4	Petter Mattei's "Love in the Time of Money" is...	positive	

Próximas etapas: [Gerar código com df](#) [Ver gráficos recomendados](#) [New interactive sheet](#)

✓ Pré-processamento de Texto

Antes de treinar um modelo de aprendizado de máquina, precisamos preparar os textos de maneira adequada. As principais etapas incluem:

1. **Remoção de tags HTML**
2. **Remoção de pontuação e números**
3. **Conversão para minúsculas**
4. **Tokenização**
5. **Remoção de stopwords**
6. **Lematização (opcional)**
7. **Vetorização com TF-IDF**

Função de limpeza do texto

```
import re
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer

# Lista de stopwords em inglês
stop_words = set(stopwords.words('english'))

def limpar_texto(texto):
    # Remove tags HTML
    texto = re.sub(r"<.*?>", "", texto)
    # Remove pontuação e números
    texto = re.sub(r"[^a-zA-Z]", " ", texto)
    # Converte para minúsculas
    texto = texto.lower()
    # Remove stopwords
    palavras = texto.split()
    palavras = [palavra for palavra in palavras if palavra not in stop_words]
    return " ".join(palavras)

# Aplicando a limpeza no dataset
df["review_limpo"] = df["review"].apply(limpar_texto)

# Visualizar exemplo
df[["review", "review_limpo"]].head(3)
```

 [nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.

	review	review_limpo	
0	One of the other reviewers has mentioned that ...	one reviewers mentioned watching oz episode ho...	
1	A wonderful little production. The...	wonderful little production filming technique ...	
2	I thought this was a wonderful way to spend ti...	thought wonderful way spend time hot summer we...	

✓ Pré-processamento de Texto

Antes de treinar um modelo de aprendizado de máquina, precisamos preparar os textos de maneira adequada. As principais etapas incluem:

1. **Remoção de tags HTML**
2. **Remoção de pontuação e números**
3. **Conversão para minúsculas**
4. **Tokenização**
5. **Remoção de stopwords**
6. **Lematização (opcional)**
7. **Vetorização com TF-IDF**

Função de limpeza do texto

```
# Limitando o número de features para acelerar o treinamento
vectorizer = TfidfVectorizer(max_features=5000)
```

```
X = vectorizer.fit_transform(df["review_limpo"])
y = df["sentiment"]
```

✓ Treinamento e Avaliação do Modelo

Agora que temos os textos vetorizados (matriz X) e os rótulos (y), vamos treinar um modelo de classificação.

Para esta tarefa, usaremos o algoritmo **Logistic Regression**, que é simples, eficiente e bastante eficaz para problemas de classificação binária como este (positivo vs. negativo).

Separando treino e teste

```
from sklearn.model_selection import train_test_split

# Separando os dados (80% treino, 20% teste)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Treinando o modelo

```
from sklearn.linear_model import LogisticRegression

# Treinamento
modelo = LogisticRegression(max_iter=200)
modelo.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=200)
```

```
from sklearn.linear_model import LogisticRegression
```

✓ Treinamento

```
modelo = LogisticRegression(max_iter=200) modelo.fit(X_train, y_train)
```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
# Predição
y_pred = modelo.predict(X_test)
```

```
# Métricas
print("Acurácia:", accuracy_score(y_test, y_pred))
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred))
```

```
Acurácia: 0.8922
```

```
Relatório de Classificação:
              precision    recall  f1-score   support

negative     0.90      0.88      0.89     4961
positive     0.88      0.91      0.89     5039
```

accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

Análise dos Resultados

Nosso modelo de classificação de sentimentos obteve os seguintes resultados na base de teste:

- **Acurácia geral:** 89,2%
- **Precisão** para classe *negativa*: 0.90
- **Recall** para classe *negativa*: 0.88
- **Precisão** para classe *positiva*: 0.88
- **Recall** para classe *positiva*: 0.91

Interpretação

- O modelo está **equilibrado** entre as duas classes, com desempenho semelhante para resenhas positivas e negativas.
- O **F1-Score** de 0.89 indica que o modelo tem **ótimo desempenho geral**, conciliando bem precisão e recall.
- A matriz de confusão indica que o modelo **comete poucos erros** de classificação, o que é importante em aplicações como:
 - Monitoramento de opinião pública
 - Análise de feedbacks de clientes
 - Gestão de reputação de marcas

Conclusão

- A análise de sentimentos é uma aplicação prática de **NLP + Classificação supervisionada**.
- Trabalhamos com um dataset real (IMDb) e aprendemos a:
 - Pré-processar texto
 - Transformar palavras em vetores com TF-IDF
 - Treinar e avaliar um modelo de machine learning
- Com uma acurácia próxima de 90%, nosso modelo está **pronto para ser aplicado em casos reais!**

Próximos passos (opcional para alunos curiosos)

- Tentar outros modelos como **Naive Bayes**, **SVM** ou **Redes Neurais**
- Usar **word embeddings** como Word2Vec ou BERT
- Classificar

✓ Previsão de Sentimento com Frases Digitadas

Vamos testar o modelo com **frases escritas manualmente** para prever se o sentimento é **positivo** ou **negativo**.

Esse tipo de interação é comum em sistemas de atendimento, redes sociais, análise de produtos, entre outros.

Reaproveitando a função de limpeza

Lembre-se: antes de prever, precisamos **aplicar o mesmo pré-processamento** utilizado no treino.

```
def prever_sentimento(texto):
    texto_limpo = limpar_texto(texto)
    vetor = vectorizer.transform([texto_limpo])
    return modelo.predict(vetor)[0]

# Entrada do usuário
entrada = input("Digite uma resenha de filme (em inglês):\n")
sentimento_previsto = prever_sentimento(entrada)

print(f"\n🔍 Sentimento previsto: {sentimento_previsto.upper()}")
```

 Digite uma resenha de filme (em inglês):
I absolutely loved this movie. It was brilliant!

 Sentimento previsto: POSITIVE

```
# Entrada do usuário
entrada = input("Digite uma resenha de filme (em inglês):\n")
sentimento_previsto = prever_sentimento(entrada)

print(f"\n🔍 Sentimento previsto: {sentimento_previsto.upper()}")
```

🔄 Digite uma resenha de filme (em inglês):
This was the worst film I've seen in years.

🔍 Sentimento previsto: NEGATIVE


```
# Entrada do usuário
entrada = input("Digite uma resenha de filme (em inglês):\n")
sentimento_previsto = prever_sentimento(entrada)

print(f"\n🔍 Sentimento previsto: {sentimento_previsto.upper()}")
```

🔄 Digite uma resenha de filme (em inglês):
Not bad at all, but it could be better.

🔍 Sentimento previsto: NEGATIVE

✅ Conclusão Final da Análise de Sentimentos

Abaixo, temos os testes feitos com frases digitadas manualmente para avaliar a capacidade preditiva do nosso modelo de análise de sentimentos:

✎ Frases testadas e resultados:

Frase em Inglês	Tradução para o Português	Sentimento Previsto	Interpretação
I absolutely loved this movie. It was brilliant!	Eu absolutamente amei este filme. Foi brilhante!	POSITIVE	✅ Correto. Sentimento fortemente positivo.
This was the worst film I've seen in years.	Este foi o pior filme que vi em anos.	NEGATIVE	✅ Correto. Crítica clara e negativa.
Not bad at all, but it could be better.	Nada mal, mas poderia ser melhor.	NEGATIVE	⚠ Interpretado como negativo, embora seja uma crítica mista com tom le

🧠 O que o modelo nos mostra?

- O modelo está **bem treinado para identificar opiniões claras**, tanto positivas quanto negativas.
- Em frases **ambíguas ou neutras**, como críticas construtivas ou "elogios com ressalvas", o modelo pode apresentar **dificuldade**, pois foi treinado apenas com as classes `positive` e `negative`.

📌 Considerações finais

- O modelo Logistic Regression com vetorização TF-IDF atingiu uma acurácia de **~89%**, o que é um ótimo resultado para um projeto introdutório de NLP.
- Projetos reais de análise de sentimentos podem se beneficiar de:
 - Inclusão de uma terceira classe: **neutro**
 - Modelos mais avançados como **BERT**, **RoBERTa** ou **transformers**
 - Uso de embeddings e redes neurais profundas

🏠 Fechamento da Aula

Você aprendeu a:

- ✅ Fazer o pré-processamento de texto
- ✅ Transformar texto em vetores com TF-IDF
- ✅ Treinar um modelo supervisionado de classificação
- ✅ Avaliar o desempenho com métricas e matriz de confusão
- ✅ Prever o sentimento de frases digitadas pelo usuário

🚀 Próximos passos sugeridos

- Tentar outros modelos (Naive Bayes, SVM, BERT)
- Expandir para 3 classes (positivo, negativo, neutro)
- Criar uma interface com Streamlit ou Flask
- Aplicar em opiniões de produtos reais, redes sociais ou pesquisas de satisfação

🗣 Discussão em Grupo: Análise de Sentimentos e NLP

Para finalizar nossa aula, reflita e debata com seus colegas sobre os seguintes temas:

1 Como você enxerga o uso da análise de sentimentos no dia a dia das empresas?

Pense em áreas como atendimento ao cliente, marketing, redes sociais, análise de produtos, política, educação, etc.

2 Quais são as limitações que um modelo de análise de sentimentos pode ter?

Reflita sobre frases ambíguas, sarcasmo, ironia, gírias, contexto cultural, idioma e possíveis consequências de decisões automáticas baseadas em sentimentos.

3 Se você fosse melhorar o modelo que construímos hoje, o que faria?

Leve em consideração o dataset, as etapas de pré-processamento, o algoritmo escolhido, ou até mesmo o tipo de saída do modelo.

 Não existe resposta certa ou errada — o objetivo é **pensar criticamente, trocar ideias e conectar o conteúdo técnico com aplicações do mundo real.**
